

A PROJECT REPORT ON

RURAL TELEMEDICINE PLATFORM WITH AI SYMPTOM TRIAGE

Domain: Healthcare / Web Application

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	31
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	32

CHAPTER I

INTRODUCTION

Access to timely medical advice is a major challenge in rural India. Primary Health Centres are often far from villages, specialists are available only in district towns, and patients may have to travel for hours, lose a day's wages and spend money on transport for a consultation that could have been handled remotely. At the same time, doctors in busy outpatient departments spend much of their time on non-urgent cases.

Patient records are usually on paper slips that are lost or not carried to the next visit, so a doctor at one clinic does not know what was prescribed at another. Elderly patients and those with chronic conditions such as diabetes and hypertension need regular follow-up, which becomes difficult when travel is required.

Telemedicine, officially recognised in India through the Telemedicine Practice Guidelines of 2020 and the eSanjeevani platform, allows doctors to consult patients through video. This project, GramaCare, is a telemedicine platform in which patients describe their symptoms in simple language (Tamil or English); an AI triage model suggests an urgency level - self-care, clinic visit or emergency - before connecting them by video to an available doctor. The doctor writes an e-prescription, and records are stored securely for follow-up. The AI supports doctors and does not replace them.

LIMITATIONS

1. Rural patients travel long distances for basic consultations.
2. Doctors lose time on non-urgent cases.
3. Patient history is not available across clinics.
4. Emergency symptoms are not always recognised quickly by patients.

1.1 History of Domain

Telemedicine began with the transmission of electrocardiograms over telephone lines in the early twentieth century and with NASA's remote monitoring of astronauts in the 1960s. In India, ISRO started a telemedicine pilot in 2001 connecting rural hospitals to

specialty hospitals via satellite, and the Apollo Telemedicine Networking Foundation established early rural telemedicine centres.

Symptom checkers such as those evaluated by Semigran et al. (2015) became popular online, and the COVID-19 pandemic accelerated telemedicine adoption. The Government of India's eSanjeevani platform has since delivered tens of crores of teleconsultations. Machine learning based triage models are now being studied to prioritise patients and support community health workers.

1.2 Overview of the Project

The patient, or a community health worker helping them, registers with a mobile number and OTP. They select or type symptoms; free-text in Tamil or English is normalised by an NLP module that maps phrases such as 'kaichal' or 'fever with chills' to standard symptom codes. Additional details such as age, duration, severity, temperature and existing conditions are collected.

A triage model (a gradient boosted classifier trained on symptom-urgency data and combined with hard red-flag rules such as chest pain with sweating or breathing difficulty) predicts the urgency level. Emergency cases are shown first-aid information and the nearest hospital and 108 ambulance number immediately. Other cases are scheduled with an available doctor for a WebRTC video consultation. The doctor sees the triage summary and history, writes an e-prescription, and the record is stored in MongoDB with the patient's consent.

1.3 Motivation

Many rural patients either ignore symptoms until they become serious or travel long distances for minor problems.

- **Motivation:** To bring a doctor to the village through the phone and help patients understand how urgently they need care.
- **Outcome:** Faster access to medical advice, prioritised emergency care, fewer unnecessary trips, and continuous patient records.

1.4 Aim of the Project

The project is entitled as “**RURAL TELEMEDICINE PLATFORM WITH AI SYMPTOM TRIAGE**”. The aim of the project is to develop a secure telemedicine web platform with AI-assisted symptom triage that prioritises patients by urgency, enables video consultations with doctors, and maintains electronic health records with consent.

1.5 Objectives

- To accept symptoms in Tamil and English and normalise them with NLP.
- To predict urgency level using a triage model with red-flag safety rules.
- To schedule appointments with available doctors.
- To provide video consultations using WebRTC.
- To issue e-prescriptions and store records securely.

1.6 Literature Review

Semigran et al. (2015): Evaluated 23 online symptom checkers and found that they listed the correct diagnosis first in 34% of cases but gave appropriate triage advice in 57% of cases, being more risk-averse for emergencies; highlighting the need for careful triage design.

Board of Governors / MoHFW (2020): The Telemedicine Practice Guidelines of India define the modes of teleconsultation, the responsibilities of registered medical practitioners and rules for e-prescriptions.

Rajkomar, Dean and Kohane (2019): Reviewed machine learning in medicine and emphasised that models should augment clinicians, be validated prospectively and be monitored for bias.

Levin et al. (2018): Showed that an electronic triage tool using machine learning in emergency departments improved differentiation of patients' acuity compared with the conventional Emergency Severity Index.

Sharma et al. (eSanjeevani, 2021): Described the national telemedicine service of India, which provides doctor-to-doctor and patient-to-doctor consultations and has enabled large-scale remote care.

W3C WebRTC 1.0 (2021): The WebRTC standard defines APIs for real-time peer-to-peer audio, video and data communication in browsers.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Currently, rural patients visit the nearest PHC or private clinic, or travel to district hospitals. Some use phone calls with doctors they know. Government telemedicine through eSanjeevani is available but patients often need help from health workers. Paper records are used for history.

DISADVANTAGES

- Long travel and waiting times.
- No prioritisation of urgent cases.
- Loss of paper records.
- Language barriers in English-only apps.
- Doctors overloaded with routine cases.

2.2 Proposed System

The proposed GramaCare platform combines AI symptom triage with safety rules, appointment scheduling, WebRTC video consultation and e-prescriptions with secure records. It supports Tamil and English and can be used by patients directly or with the help of village health workers.

ADVANTAGES

- Urgency-based prioritisation; emergencies are highlighted immediately.
- Consultation from home or village centre.
- Records available at every consultation.
- Bilingual interface.
- Doctors see a structured summary before the call.

2.3 Project Schedule

The development of GramaCare follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the GramaCare system are listed below.

- **Registration:** Patient registers using mobile number and OTP.
- **Symptom entry:** Select or type symptoms in Tamil or English.
- **Symptom normalisation:** Map free text to standard symptom codes.
- **Triage:** Predict urgency: self-care, clinic, emergency.
- **Emergency guidance:** Show first aid, nearest hospital and 108 number.
- **Appointment:** Book slot with available doctor.
- **Video consultation:** WebRTC call between patient and doctor.
- **E-prescription and records:** Doctor issues prescription; records stored with consent.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Safety:** Red-flag rules always override the model to emergency.
- **Security:** JWT authentication, HTTPS, encrypted records.

- **Privacy:** Consent before storing or sharing records.
- **Performance:** Video works on 4G at 300 kbps adaptive.
- **Usability:** Large icons, Tamil text, voice input option.
- **Availability:** 99% uptime during service hours.

3.3 User Requirements

- Easy symptom entry in Tamil
- Clear advice on urgency
- Quick appointment with a doctor
- Video call without installing apps
- Prescription available on phone

3.4 System Requirements

- Node.js server with Express
- MongoDB database
- Python triage microservice
- WebRTC signalling server (Socket.IO) and TURN server
- React front end

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i5 Processor 2.4 GHz or above
Hard Disk	: 256 GB SSD
RAM	: 8 GB
Camera and microphone	: Webcam / smartphone camera
Monitor	: 15" colour monitor
Optional devices	: Digital thermometer, BP monitor, pulse oximeter at village centre

4.2 Software Specification

Operating System	: Windows 10 / Ubuntu 22.04
Coding Language	: JavaScript (Node.js, React), Python 3.10
Video	: WebRTC, Socket.IO signalling, coturn TURN server
Database	: MongoDB
ML	: scikit-learn / XGBoost triage model
Authentication	: JWT, OTP via SMS

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed GramaCare system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

WebRTC is supported by all modern browsers, and Node.js with Socket.IO provides signalling. The triage model is a lightweight classifier with rule-based safety checks. The project is technically feasible.

ECONOMIC FEASIBILITY

Open-source software and low-cost hosting keep costs low. Patients save travel costs and wages. It is economically feasible.

OPERATIONAL FEASIBILITY

Patients can use the platform with the help of village health workers; doctors use a simple web dashboard. The workflow follows the national Telemedicine Practice Guidelines. It is operationally feasible.

5.2 About the Software

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a

single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

NODE.JS AND EXPRESS

Node.js is an open-source, cross-platform JavaScript runtime environment built on Chrome's V8 engine. It uses an event-driven, non-blocking I/O model that makes it lightweight and efficient for data-intensive real-time applications.

Express.js is a minimal and flexible Node.js web framework that provides a robust set of features for building REST APIs, including routing, middleware and request/response handling. Together they allow both front end and back end to be written in JavaScript.

- Handles many concurrent connections
- npm - the largest package registry
- Ideal for real-time and IoT back ends

WEBRTC

WebRTC (Web Real-Time Communication) is a free, open standard that enables web browsers and mobile applications to make peer-to-peer audio and video calls without plugins.

WebRTC handles echo cancellation, bandwidth adaptation and encryption (DTLS-SRTP) automatically, so a video consultation can work even on a slower 4G connection.

MONGODB

MongoDB is a source-available, cross-platform, document-oriented NoSQL database. It stores data as flexible JSON-like documents (BSON), which means fields can vary from document to document and the data structure can be changed over time.

MongoDB supports indexing, aggregation pipelines, geospatial queries and time-series collections, which makes it well suited for storing sensor readings, user profiles and application logs.

- Schema-less flexible documents
- Horizontal scaling through sharding
- Powerful aggregation framework

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

XGBOOST

XGBoost (Extreme Gradient Boosting) is an optimised, distributed gradient boosting library designed to be highly efficient, flexible and portable. It implements machine learning algorithms under the gradient boosting framework, where many shallow decision trees are added one after another and each new tree corrects the errors of the previous ones.

XGBoost handles missing values automatically, supports regularisation to avoid over-fitting, and allows class weighting through the `scale_pos_weight` parameter, which makes it very suitable for imbalanced problems such as fraud and fault detection on tabular data.

- High accuracy on tabular data
- Fast training with parallel tree construction
- Feature importance for explainability

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of GramaCare is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

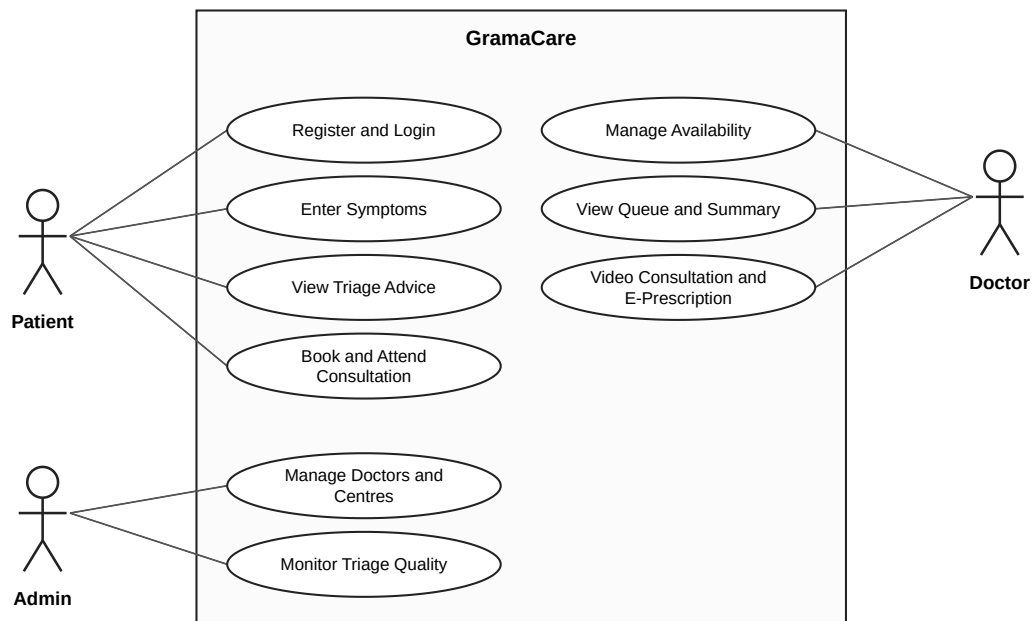


Fig 1: Use case diagram of GramaCare

6.2 System Architecture

GramaCare uses a MERN-style architecture with a separate Python triage microservice and WebRTC media flowing directly between browsers (or through a TURN relay when needed).

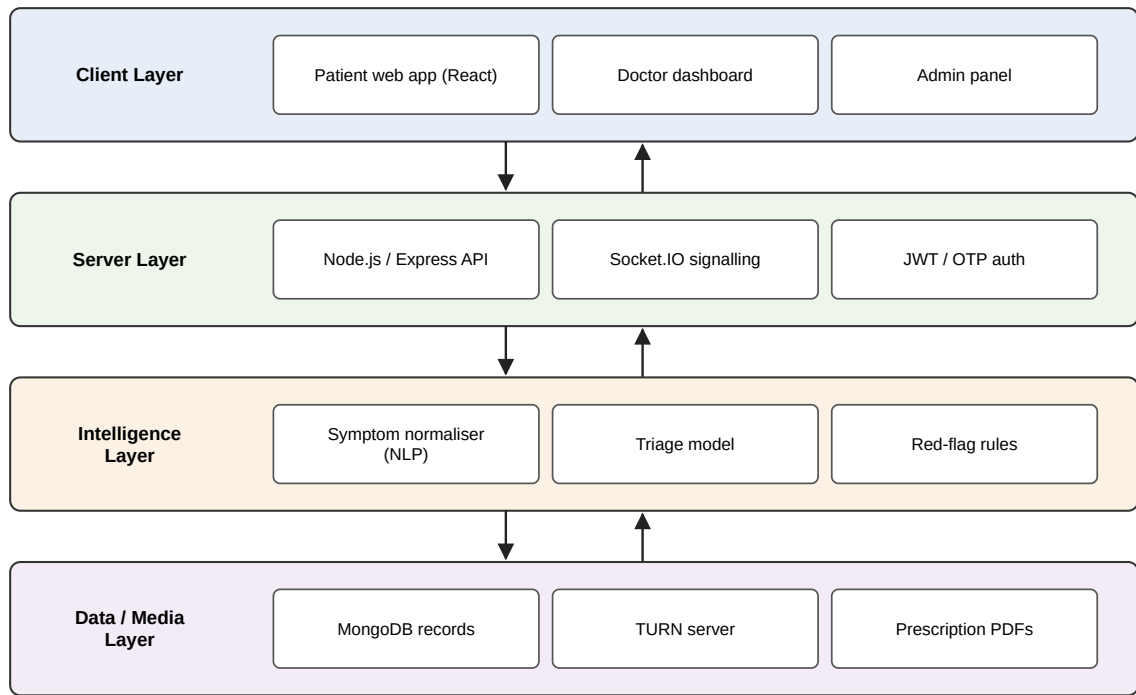


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

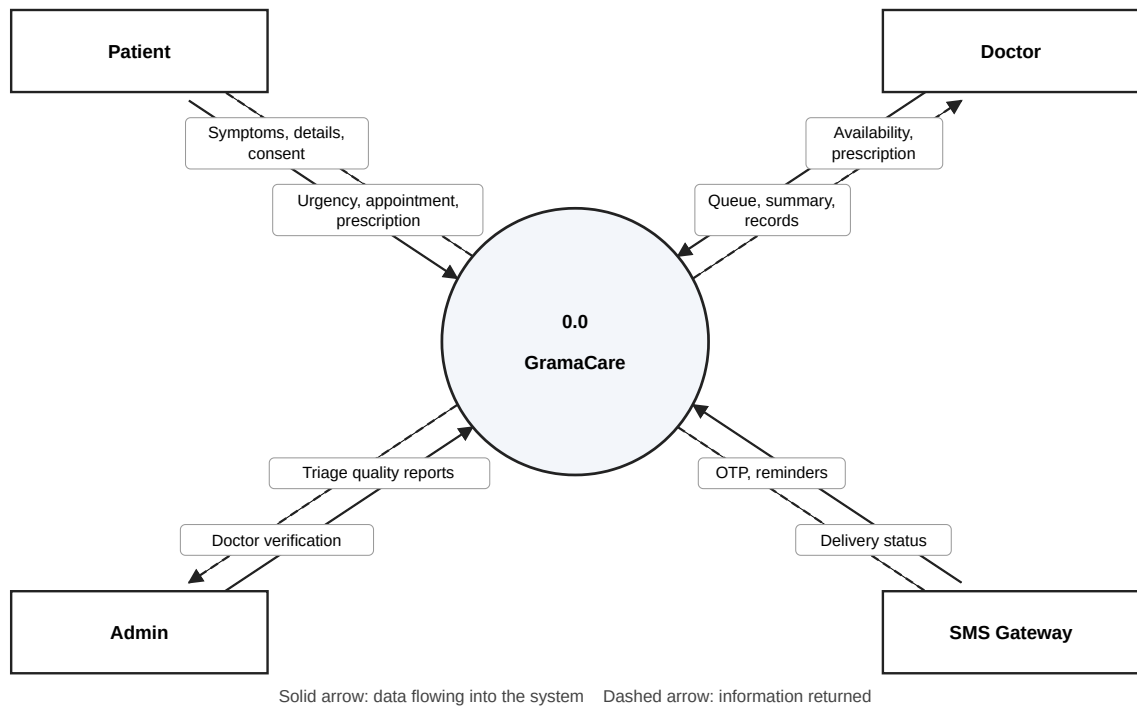


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

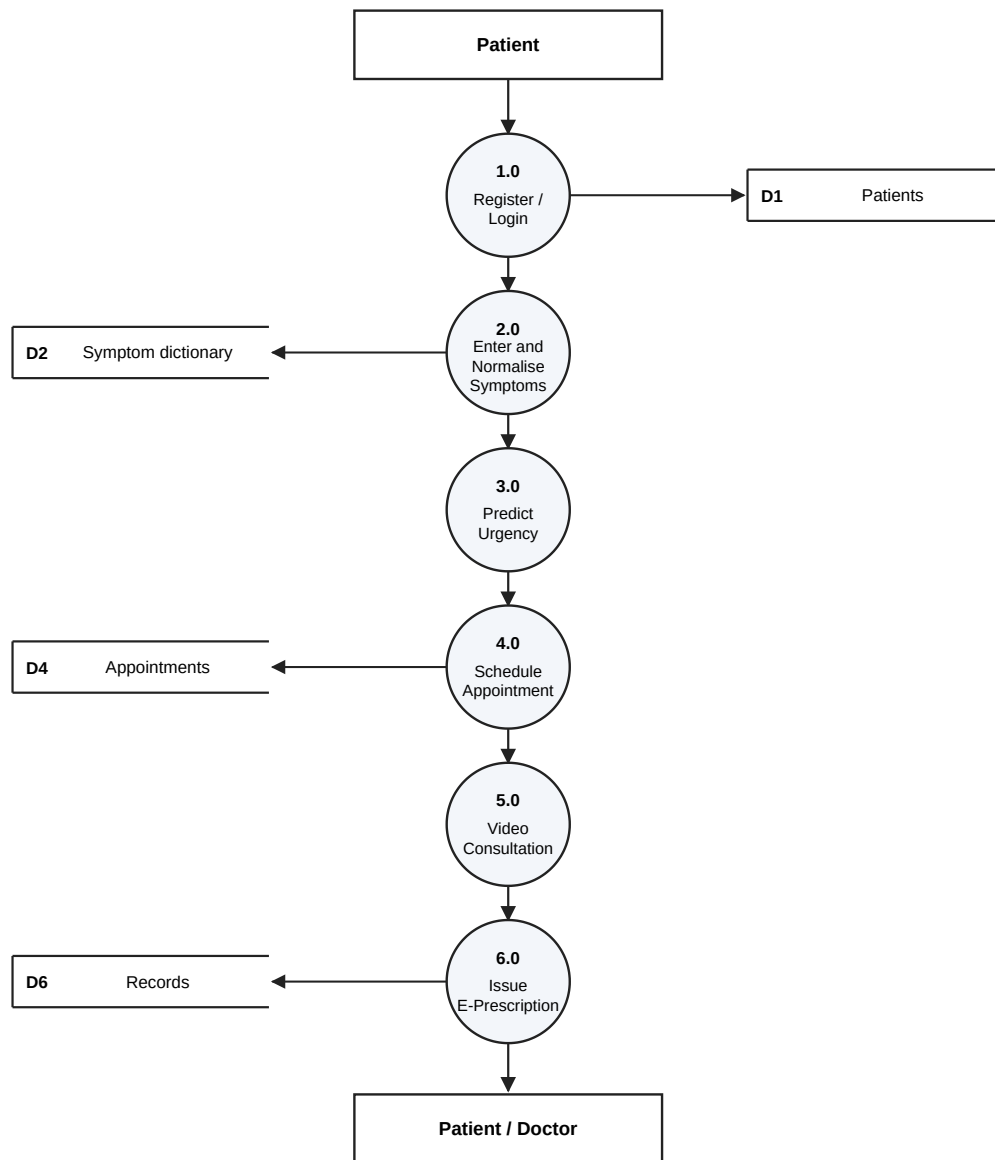


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

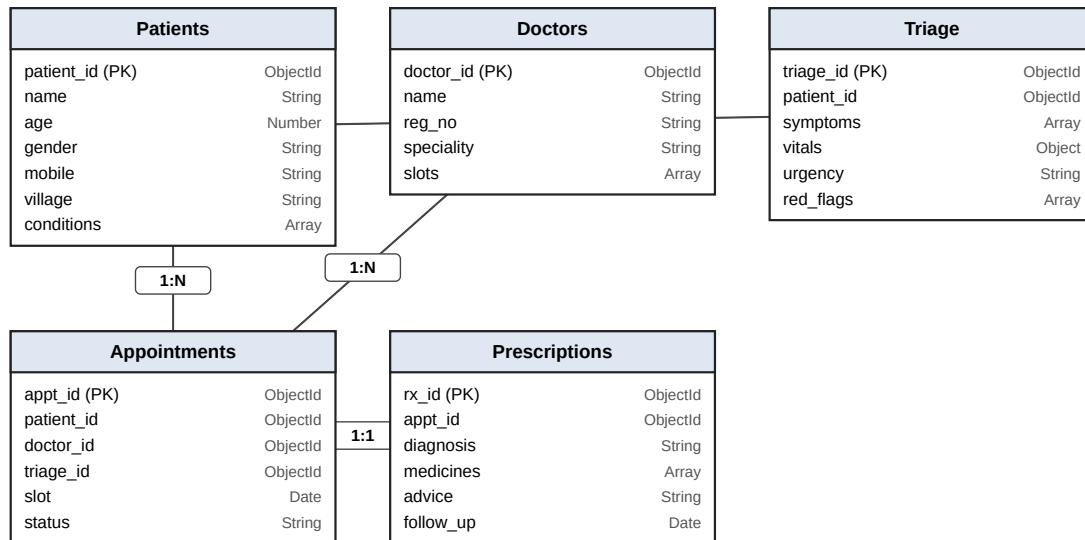


Fig 5: Entity relationship diagram

6.5 Table Design

PATIENTS

FIELD NAME	DATA TYPE	DESCRIPTION
patient_id	ObjectId	Primary key
name	String	Patient name
age	Number	Age
gender	String	Gender
mobile	String	Mobile (OTP verified)
village	String	Village
conditions	Array	Chronic conditions

DOCTORS

FIELD NAME	DATA TYPE	DESCRIPTION
doctor_id	ObjectId	Primary key
name	String	Doctor name
reg_no	String	Medical council registration
speciality	String	Speciality
slots	Array	Available slots

TRIAGE

FIELD NAME	DATA TYPE	DESCRIPTION
triage_id	ObjectId	Primary key
patient_id	ObjectId	Foreign key to Patients
symptoms	Array	Normalised symptom codes
vitals	Object	Temperature, SpO2 etc.
urgency	String	self_care / clinic / emergency
red_flags	Array	Triggered rules

APPOINTMENTS

FIELD NAME	DATA TYPE	DESCRIPTION
appt_id	ObjectId	Primary key
patient_id	ObjectId	Foreign key
doctor_id	ObjectId	Foreign key
triage_id	ObjectId	Foreign key
slot	Date	Scheduled time
status	String	booked / done / missed

PRESCRIPTIONS

FIELD NAME	DATA TYPE	DESCRIPTION
rx_id	ObjectId	Primary key
appt_id	ObjectId	Foreign key to Appointments
diagnosis	String	Provisional diagnosis
medicines	Array	Name, dose, frequency, days
advice	String	Advice
follow_up	Date	Follow-up date

6.6 Algorithm Design

The triage algorithm combines deterministic safety rules with a machine learning classifier so that dangerous symptoms are never under-triaged.

1. **Patient registers and enters symptoms** – Symptoms are selected or typed; follow-up questions collect duration, severity and vitals.
2. **Symptoms normalized with NLP** – A Tamil-English dictionary and fuzzy matching (RapidFuzz) map phrases to symptom codes.
3. **Triage model predicts urgency level** – An XGBoost classifier uses symptom codes, age, duration, severity and vitals to predict the urgency class.
4. **Emergency cases are highlighted with first-aid and hospital info** – Red-flag rules (e.g., chest pain + sweating, breathing difficulty, SpO2 < 92, unconsciousness, severe bleeding, stroke signs) force Emergency regardless of the model.
5. **Appointment is scheduled with available doctor** – Clinic-level cases get the earliest slot; self-care cases get advice and an optional slot.
6. **Video consultation via WebRTC** – Peer-to-peer video with Socket.IO signalling and TURN fallback.
7. **Doctor writes e-prescription** – Prescription follows the format in the Telemedicine Practice Guidelines and is generated as a PDF.
8. **Records stored securely with consent** – Records are encrypted and visible only to the patient and treating doctors.

PSEUDO CODE

```
TRIAGE(patient, text, answers):  
  codes <- NORMALISE(text)  // dictionary + fuzzy match, Tamil and English  
  x <- ENCODE(codes, patient.age, answers.duration, answers.severity,  
  answers.vitals)  
  flags <- [rule for rule in RED_FLAG_RULES if rule(codes, answers.vitals)]  
  IF flags NOT EMPTY THEN RETURN EMERGENCY, flags  
  probs <- XGB.predict_proba(x)  
  IF probs[EMERGENCY] > 0.25 THEN RETURN EMERGENCY      // conservative cut-off  
  RETURN argmax(probs), []
```

6.7 Modules

The modules are organised according to the Patient, Doctor and Admin roles.

MODULES

PATIENT

1. REGISTER AND LOGIN
2. ENTER SYMPTOMS
3. VIEW TRIAGE ADVICE
4. BOOK AND ATTEND CONSULTATION

DOCTOR

1. MANAGE AVAILABILITY
2. VIEW QUEUE AND SUMMARY
3. VIDEO CONSULTATION AND E-PRESCRIPTION

ADMIN

1. MANAGE DOCTORS AND CENTRES
2. MONITOR TRIAGE QUALITY

PATIENT

REGISTER AND LOGIN

The patient registers with name, age, gender, village and mobile number verified by OTP. A health worker can register on behalf of a patient.

ENTER SYMPTOMS

The patient selects symptoms from icons or types them in Tamil/English, and answers follow-up questions on duration, severity and temperature.

VIEW TRIAGE ADVICE

The urgency level is shown with advice. For emergencies, first aid steps, the 108 ambulance number and the nearest hospital are shown immediately.

BOOK AND ATTEND CONSULTATION

The patient books a slot and joins the video call at the scheduled time; the prescription is available afterwards.

DOCTOR

MANAGE AVAILABILITY

Doctors set consultation slots.

VIEW QUEUE AND SUMMARY

Doctors see the patient queue ordered by urgency with a structured triage summary and past records.

VIDEO CONSULTATION AND E-PRESCRIPTION

Doctors consult by video and write a prescription with medicines, dosage, advice and follow-up date.

ADMIN

MANAGE DOCTORS AND CENTRES

Admin verifies doctors' registration numbers and manages village centres.

MONITOR TRIAGE QUALITY

Admin compares AI triage with doctors' final urgency assessment to monitor and improve the model.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

GramaCare was implemented using React for the front end, Node.js with Express for the API, Socket.IO for WebRTC signalling and a coturn TURN server for patients behind restrictive networks. The triage model is a Python FastAPI microservice. MongoDB stores all records.

7.2 Implementation

The triage model was trained on a symptom-urgency dataset prepared from public symptom-disease datasets mapped to urgency levels, and reviewed by a physician advisor. Red-flag rules were taken from standard emergency triage guidance. The model's conservative threshold for emergencies was chosen to maximise emergency recall.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	OTP login	Valid mobile + OTP	Logged in	Pass
TC02	Tamil symptom	'காய்ச்சல் மூன்று நாள்'	Mapped to fever, duration 3 days	Pass
TC03	Self-care case	Mild cold, 1 day, adult	Self-care advice	Pass
TC04	Red-flag	Chest pain + sweating, age 58	EMERGENCY with 108 and hospital	Pass
TC05	Low SpO2	SpO2 = 89%	EMERGENCY	Pass
TC06	Appointment	Book 10:30 slot	Slot reserved, SMS sent	Pass
TC07	Video call on 4G	Patient on mobile data	Call connects via TURN, adaptive quality	Pass
TC08	E-prescription	Doctor submits Rx	PDF available to patient	Pass

PERFORMANCE EVALUATION

Triage evaluation on 600 vignettes reviewed by a physician:

Urgency class	Precision	Recall
Self-care	0.86	0.82
Clinic	0.81	0.84
Emergency	0.79	0.98
Overall accuracy	83.5%	-

7.4 Source Code

triage_service.py

```
import json, joblib, numpy as np
from fastapi import FastAPI
from pydantic import BaseModel
```

```

from rapidfuzz import process, fuzz

app = FastAPI()
model = joblib.load("models/triage_xgb.pkl")
SYMPTOMS = json.load(open("data/symptom_dictionary.json")) # {"fever":
["fever", "kaichal", "காய்ச்சல்"], ...}
CODES = sorted(SYMPTOMS)
PHRASES = {p: code for code, ps in SYMPTOMS.items() for p in ps}
LABELS = ["self_care", "clinic", "emergency"]

RED_FLAGS = [
    ("Chest pain with sweating", lambda s, v: {"chest_pain", "sweating"} <= s),
    ("Breathing difficulty", lambda s, v: "breathlessness" in s and
v.get("severity", 0) >= 2),
    ("Low oxygen", lambda s, v: v.get("spo2", 100) < 92),
    ("Stroke signs", lambda s, v: bool({"face_droop", "slurred_speech",
"one_side_weakness"} & s)),
    ("Unconscious / seizure", lambda s, v: bool({"unconscious", "seizure"} &
s)),
    ("Severe bleeding", lambda s, v: "bleeding" in s and v.get("severity", 0) ==
3),
    ("High fever in infant", lambda s, v: "fever" in s and v.get("age", 30) <
1),
]

class Request(BaseModel):
    text: str
    selected: list[str] = []
    age: int
    duration_days: float
    severity: int # 1 mild, 2 moderate, 3 severe
    temperature: float | None = None
    spo2: int | None = None

def normalise(text):
    found = set()
    for part in text.lower().replace(" ", " ").split(" "):
        m = process.extractOne(part.strip(), PHRASES.keys(),
scorer=fuzz.partial_ratio)
        if m and m[1] >= 85:
            found.add(PHRASES[m[0]])
    return found

@app.post("/triage")
def triage(r: Request):
    s = normalise(r.text) | set(r.selected)
    vit = {"age": r.age, "severity": r.severity, "spo2": r.spo2 or 100}
    flags = [name for name, rule in RED_FLAGS if rule(s, vit)]
    if flags:
        return {"symptoms": sorted(s), "urgency": "emergency", "red_flags":
flags}
    x = np.array([[1 if c in s else 0 for c in CODES] +
[r.age, r.duration_days, r.severity, r.temperature or 37.0,
r.spo2 or 98]])
    p = model.predict_proba(x)[0]
    urgency = "emergency" if p[2] > 0.25 else LABELS[int(np.argmax(p[:2]))]
    return {"symptoms": sorted(s), "urgency": urgency, "probabilities":
dict(zip(LABELS, p.round(3).tolist()))}

```

signalling.js (WebRTC signalling with Socket.IO)

```
const express = require("express");
const http = require("http");
const jwt = require("jsonwebtoken");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

io.use((socket, next) => {
  try { socket.user = jwt.verify(socket.handshake.auth.token,
    process.env.JWT_SECRET); next(); }
  catch (e) { next(new Error("unauthorised")); }
});

io.on("connection", socket => {
  socket.on("join", async ({ appointmentId }) => {
    // only the patient and doctor of this appointment may join
    const appt = await require("./db").appointments.findOne({ _id: appointmentId });
    const allowed = [String(appt.patient_id),
      String(appt.doctor_id)].includes(socket.user.id);
    if (!allowed) return socket.emit("error", "Not part of this consultation");
    socket.join(appointmentId);
    socket.to(appointmentId).emit("peer-joined", { role: socket.user.role });
  });
  // relay SDP offers/answers and ICE candidates between the two peers
  for (const ev of ["offer", "answer", "ice-candidate"]) {
    socket.on(ev, ({ appointmentId, data }) => socket.to(appointmentId).emit(ev,
      data));
  }
  socket.on("end-call", ({ appointmentId }) => io.to(appointmentId).emit("call-
    ended"));
});

server.listen(5001, () => console.log("Signalling server running on 5001"));
```

client/useCall.js (React hook)

```
import { io } from "socket.io-client";

export async function startCall(appointmentId, token, localVideo, remoteVideo) {
  const socket = io("https://signal.gramacare.in", { auth: { token } });
  const pc = new RTCPeerConnection({ iceServers: [
    { urls: "stun:stun.l.google.com:19302" },
    { urls: "turn:turn.gramacare.in:3478", username: "gc", credential: "secret"
  } ] });
  const stream = await navigator.mediaDevices.getUserMedia({ video: { width: 640
  }, audio: true });
  localVideo.srcObject = stream;
  stream.getTracks().forEach(t => pc.addTrack(t, stream));
  pc.ontrack = e => (remoteVideo.srcObject = e.streams[0]);
  pc.onicecandidate = e => e.candidate && socket.emit("ice-candidate", {
    appointmentId, data: e.candidate });

  socket.on("peer-joined", async () => {
    const offer = await pc.createOffer(); await pc.setLocalDescription(offer);
    socket.emit("offer", { appointmentId, data: offer });
  });
}
```

```

});
socket.on("offer", async o => {
  await pc.setRemoteDescription(o);
  const ans = await pc.createAnswer(); await pc.setLocalDescription(ans);
  socket.emit("answer", { appointmentId, data: ans });
});
socket.on("answer", a => pc.setRemoteDescription(a));
socket.on("ice-candidate", c => pc.addIceCandidate(c));
socket.emit("join", { appointmentId });
return () => { stream.getTracks().forEach(t => t.stop()); pc.close();
socket.disconnect(); };
}

```

7.5 Screen Layout

SCREENSHOTS:

SYMPTOM ENTRY

The screenshot shows a web application interface for 'GramaCare'. The header is green with the logo and navigation links: Symptoms, Appointments, Consult, Records, and Doctor. The main content area is titled 'உங்கள் அறிகுறிகள் / Your symptoms'. It contains a form with the following fields:

- Describe your problem:** A text input field containing 'காய்ச்சல் மூன்று நாள், உடல் வலி, இருமல்'.
- Since how many days?:** A text input field containing '3 days'.
- How severe?:** A text input field containing 'Moderate'.
- Temperature (if known):** A text input field containing '101.2 F'.
- Existing conditions:** A text input field containing 'Diabetes'.

At the bottom of the form is a green button labeled 'Check Urgency'. To the right of the form is a 'Note' section with three lines of text:

- This tool helps decide how soon you should see a doctor.
- It does not replace a doctor's diagnosis.
- In an emergency call 108 immediately.

TRIAGE RESULT - CLINIC

https://gramacare.app/

GramaCare

[Symptoms](#)[Appointments](#)[Consult](#)[Records](#)[Doctor](#)

Triage result

See a doctor within 24 hours

Symptoms understood	Fever (3 days), body ache, cough
Risk factors	Diabetes
Suggested	Video consultation with General Physician
Next available	Today 4:30 PM - Dr. S. Meena

Drink plenty of fluids and rest.

Paracetamol may be taken for fever as per label.

If breathing difficulty develops, call 108.

TRIAGE RESULT - EMERGENCY

https://gramacare.app/

GramaCare

[Symptoms](#)[Appointments](#)[Consult](#)[Records](#)[Doctor](#)

Triage result

EMERGENCY - Call 108 now

Red flags	Chest pain with sweating
Nearest hospital	Govt Hospital, Papanasam (6 km)
Ambulance	108

Make the person sit and rest; loosen tight clothing.

Do not give food or water.

Doctor has been notified and will call you.

DOCTOR DASHBOARD - QUEUE

https://gramacare.app/

GramaCare

SymptomsAppointmentsConsultRecordsDoctor

Today's queue - Dr. S. Meena

Time	Patient	Age	Triage	Main symptoms	Status
Now	Murugesan	58	EMERGENCY	Chest pain, sweating	Calling
4:30	Lakshmi	42	CLINIC	Fever 3 d, cough (diabetic)	Waiting
4:45	Arjun	7	CLINIC	Ear pain, fever	Booked
5:00	Selvi	35	SELF-CARE	Mild cold	Booked

E-PRESCRIPTION

https://gramacare.app/

GramaCare

SymptomsAppointmentsConsultRecordsDoctor

E-Prescription - Lakshmi (42 F)

Rx #GC-2026-10-0481

Provisional diagnosis	Acute febrile illness with URTI
Medicines	Tab Paracetamol 500 mg - 1-1-1 x 3 days
	Syr cough suppressant 10 ml - 0-0-1 x 5 days
Advice	Check blood sugar daily; fluids; CBC if fever > 5 days
Follow-up	17-10-2026
Doctor	Dr. S. Meena, MBBS (Reg. TN-87214)

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Rural Telemedicine Platform with AI Symptom Triage brings timely medical advice to villages by combining bilingual symptom entry, AI-assisted urgency triage with strict red-flag safety rules, WebRTC video consultation and e-prescriptions. The triage module achieved 98% recall for emergency cases on physician-reviewed vignettes, ensuring that dangerous symptoms are not missed.

Doctors receive a prioritised queue with structured summaries, reducing consultation time, while patients save travel time and cost and retain their records for follow-up. The platform supports doctors rather than replacing them, in line with India's Telemedicine Practice Guidelines.

8.2 Future Enhancement

- Voice-based symptom entry in Tamil for low-literacy users.
- Integration with ABHA (Ayushman Bharat Health Account) for interoperable records.
- Bluetooth medical devices to send vitals automatically.
- Offline-first mode for areas with poor connectivity.
- Medicine delivery and pharmacy integration.

CHAPTER IX

BIBLIOGRAPHY

1. Semigran, H. L., Linder, J. A., Gidengil, C., and Mehrotra, A., "Evaluation of symptom checkers for self diagnosis and triage: audit study", BMJ, 351:h3480, 2015.
2. Board of Governors in supersession of the Medical Council of India, "Telemedicine Practice Guidelines", Ministry of Health and Family Welfare, Government of India, 2020.
3. Rajkomar, A., Dean, J., and Kohane, I., "Machine Learning in Medicine", New England Journal of Medicine, Vol. 380, pp. 1347-1358, 2019.
4. Levin, S. et al., "Machine-Learning-Based Electronic Triage More Accurately Differentiates Patients With Respect to Clinical Outcomes Compared With the Emergency Severity Index", Annals of Emergency Medicine, Vol. 71, No. 5, 2018.
5. W3C, "WebRTC 1.0: Real-Time Communication Between Browsers", W3C Recommendation, 2021.
6. Chen, T., and Guestrin, C., "XGBoost: A Scalable Tree Boosting System", Proc. ACM SIGKDD, 2016.

WEBSITE REFERENCE

1. <https://esanjeevani.mohfw.gov.in>
2. <https://webrtc.org>
3. <https://socket.io>
4. <https://www.mongodb.com/docs>